SESIÓN 08

Guía Práctica 08: Integración Multicapa y Desarrollo Web Empresarial con Servlets HTTP y Páginas JSP (Semana 8)

1. MARCO CONCEPTUAL Y OBJETIVOS DE APRENDIZAJE

> En esta práctica de cierre consolidamos la arquitectura multicapa conectando la interfaz de usuario de escritorio con la lógica del negocio y la base de datos MySQL. Adicionalmente, migramos la aplicación al entorno web desplegando un proyecto Jakarta EE en el servidor Apache Tomcat. Desarrollamos un Servlet controlador con la anotación `@WebServlet` sobrescribiendo `doGet` para entregar el formulario HTML y `doPost` para procesar el inicio de sesión web, integrando además la renderización dinámica con JavaServer Pages (JSP).

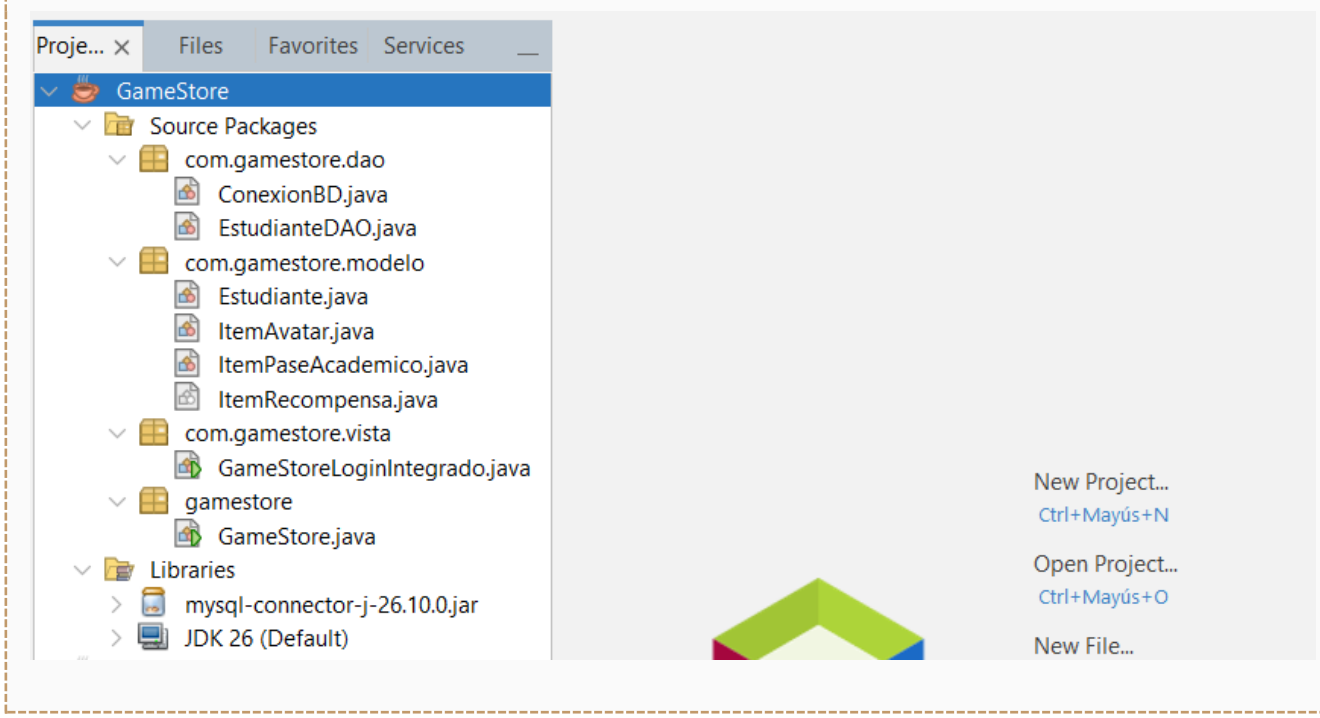
2. PROCEDIMIENTO TÉCNICO, CÓDIGO FUENTE Y EVIDENCIAS VISUALES

PASO 1: ORGANIZACIÓN DEL PROYECTO EN CAPAS INDEPENDIENTES DE SOFTWARE

Dividimos las clases de la solución en paquetes especializados: **com.gamestore.modelo** para representar los objetos de dominio, **com.gamestore.dao** para la persistencia con MySQL y **com.gamestore.vista** para formularios de escritorio y Servlets web.

Código fuente desarrollado para el laboratorio:

```
01  package com.gamestore.modelo;      // Clases: ItemRecompensa, ItemAvatar, Estudiante // Paquete para las clases del modelo de objetos
02  package com.gamestore.dao;         // Clases: ConexionBD, EstudianteDAO // Paquete para la persistencia y conexión a MySQL
03  package com.gamestore.vista;       // Clases: GameStoreLoginIntegrado, Servlets Web // Paquete para la interfaz gráfica y controladores
```

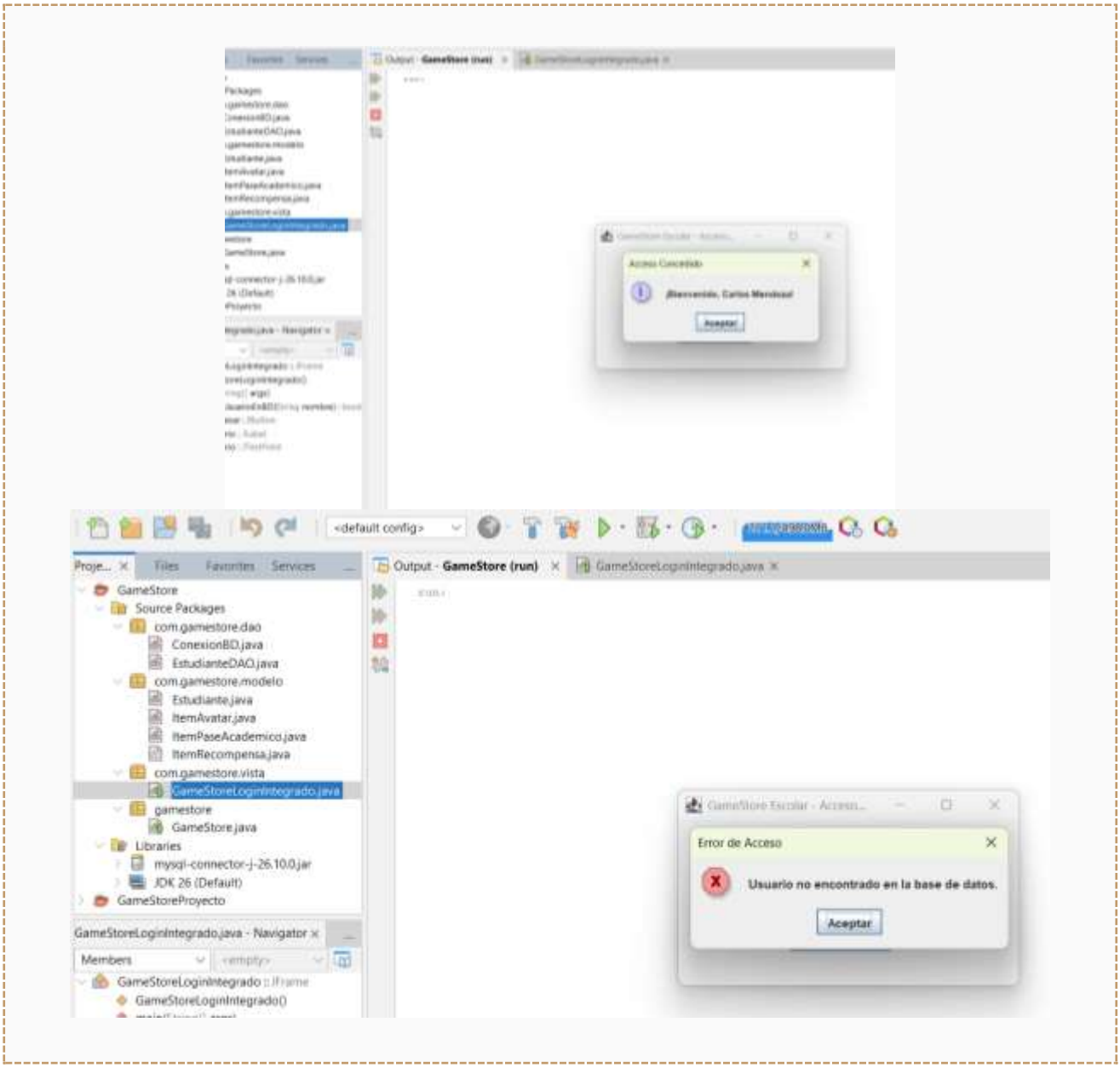


PASO 2: AUTENTICACIÓN DE CREDENCIALES EN MYSQL DESDE LA INTERFAZ GRÁFICA

Conectamos la interfaz gráfica de acceso con la base de datos validando el nombre ingresado mediante el método `validarUsuarioEnBD(...)`, el cual ejecuta una búsqueda con **PreparedStatement** sobre la tabla estudiantes.

Código fuente desarrollado para el laboratorio:

```
01      private boolean validarUsuarioEnBD(String nombre) { // Método de validación contra MySQL
02          boolean existe = false; // Variable de estado de autenticación
03          String sql = "SELECT * FROM estudiantes WHERE nombre = ?"; // Consulta SQL de búsqueda
parametrizada
04          try (Connection conn = ConexionBD.obtenerConexion(); // Apertura del enlace JDBC
05               PreparedStatement pstmt = conn.prepareStatement(sql)) { // Preparación de la sentencia
precompilada
06              pstmt.setString(1, nombre); // Asignación del nombre ingresado
07              try (ResultSet rs = pstmt.executeQuery()) { // Ejecución de la consulta SQL
08                  if (rs.next()) { // Verificación de coincidencia en la base de datos
09                      existe = true; // Confirmación de existencia del usuario
10                  } // Cierre del condicional
11              } // Cierre del flujo ResultSet
12          } catch (SQLException e) { // Manejo de excepciones de base de datos
13              System.out.println("Error al validar usuario: " + e.getMessage()); // Reporte de error por
consola
14          } // Cierre del bloque de captura
15          return existe; // Retorno de la respuesta de autenticación
16      } // Fin del método de validación
```



PASO 3: IMPLEMENTACIÓN DEL SERVLET CONTROLADOR DE ACCESO WEB EN APACHE TOMCAT

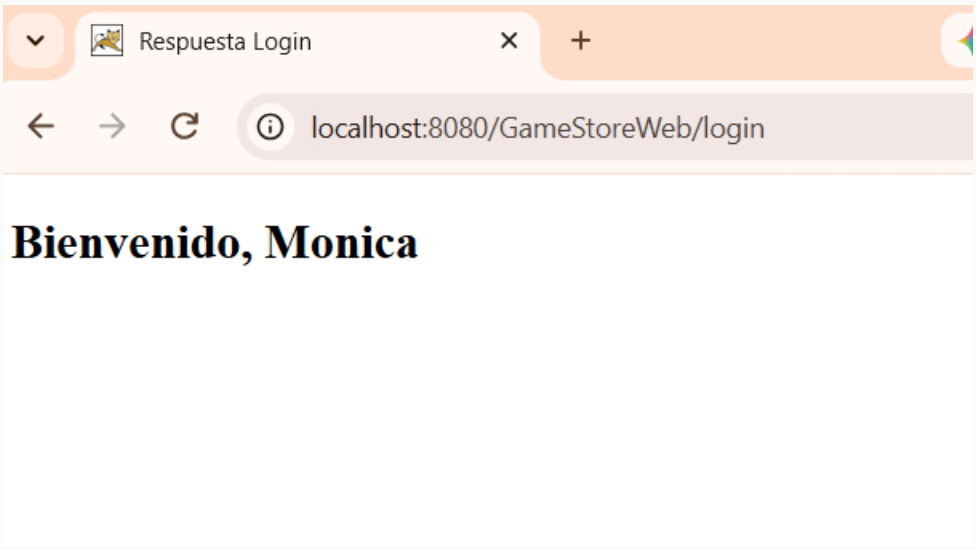
Programamos **GameStoreLoginServlet** heredando de **HttpServlet** con la anotación `@WebServlet("/login")`. Implementamos **doGet** para presentar el formulario web y **doPost** para procesar los datos recibidos del navegador.

Código fuente desarrollado para el laboratorio:

```
01  import java.io.IOException; import java.io.PrintWriter; // Importación de librerías para el manejo de flujos de salida
02  import jakarta.servlet.ServletException; import jakarta.servlet.annotation.WebServlet; // Importación del API de Servlets
03  import jakarta.servlet.http.*; // Librerías para procesamiento de solicitudes HTTP
04  @WebServlet("/login") // Ruta de mapeo de la aplicación en Apache Tomcat
05  public class GameStoreLoginServlet extends HttpServlet { // Declaración de la clase Servlet controlador
06      @Override protected void doGet(HttpServletRequest request, HttpServletResponse response)
```

```
throws ServletException, IOException { // Atención a solicitudes HTTP GET
07          response.setContentType("text/html;charset=UTF-8"); // Configuración del formato de respuesta HTML
08          try (PrintWriter out = response.getWriter()) { // Apertura del canal de salida de texto
09              out.println("<html><body>"); // Inicio de la estructura HTML
10              out.println("<h2>GameStore Escolar - Acceso Web</h2>"); // Cabecera del portal web
11              out.println("<form method='post' action='login'>"); // Formulario web con envío POST
12              out.println("Usuario: <input type='text' name='usuario'><br>"); // Campo de entrada para el
usuario
13              out.println("<input type='submit' value='Ingresar'>"); // Botón de envío de formulario
14              out.println("</form></body></html>"); // Cierre del código HTML
15          } // Cierre del canal de salida
16      } // Fin del método doGet
17      @Override protected void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException { // Atención a solicitudes HTTP POST
18          String usuario = request.getParameter("usuario"); // Captura del parámetro enviado en la petición
19          response.setContentType("text/html;charset=UTF-8"); // Configuración del formato de salida
20          try (PrintWriter out = response.getWriter()) { // Apertura del canal de escritura HTML
21              out.println("<html><body>"); // Cuerpo de la respuesta
22              if (usuario != null && !usuario.trim().isEmpty()) { // Validación del dato recibido
23                  out.println("<h2>Bienvenido, " + usuario + "</h2>"); // Mensaje de bienvenida al portal
24              } else { // Rama alternativa para campo vacío
25                  out.println("<h2>Error: ingrese un usuario</h2>"); // Mensaje de advertencia
26              } // Cierre de la validación
27              out.println("</body></html>"); // Cierre del cuerpo HTML
28          } // Cierre del flujo de respuesta
29      } // Fin del método doPost
30  } // Fin de la clase Servlet
```





3. GLOSARIO CONCEPTUAL Y TERMINOLOGÍA TÉCNICA

Concepto / Término	Descripción y rol técnico
Arquitectura Multicapa	Patrón que separa la presentación, la lógica del negocio y la persistencia en niveles independientes.
`Servlet`	Clase escrita en Java ejecutada dentro de un servidor web para responder solicitudes HTTP.
`Apache Tomcat`	Servidor de aplicaciones open-source diseñado para procesar Servlets y páginas JSP.
JSP (JavaServer Pages)	Tecnología web que permite la inserción dinámica de código Java en páginas HTML.
`doGet / doPost`	Métodos de HttpServlet encargados de gestionar peticiones HTTP de lectura (GET) o envío (POST).
HttpServletRequest / Response	Objetos que encapsulan la información enviada por el cliente y la respuesta del servidor.